705.641.81: Natural Language Processing Self-Supervised Models

Homework 6: Adapting Transformers Language Models for Your Problems

For homework deadline and collaboration policy, please see our Canvas page.

Name: ____Rajiv Khandpur_____

Collaborators, if any: _____

Sources used for your homework, if any:

I used Claude to assist with some of the programming problems.

This assignment will return to basics of Transformers and dig deeper into the fundamental concepts. Additionally, we will get our hands dirty by fine-tuning Transformer language models and comparing their performance.

Homework goals: After completing this homework, you should be comfortable with:

- Training a pre-trained Language Model for your tasks
- You will explore different variations of model training
- You will compare the above results with few-shot prompting of models.

Concepts, intuitions and big picture

1. How does the BERT model expect a pair of sentences to be processed?

- ☐ `Tokens-of-sentence-1 [SEP] Tokens-of-sentence-2`
- ☐ `[CLS] Tokens-of-sentence-1 Tokens-of-sentence-2`
- ☑ `[CLS] Tokens-of-sentence-1 [SEP] Tokens-of-sentence-2`

2. When should you train a new tokenizer?

- ☐ When your dataset is similar to that used by an existing pretrained model, and you want to pretrain a new model
- ☐ When your dataset is similar to that used by an existing pretrained model, and you want to fine-tune a new model using this pretrained model
- ☑ When your dataset is different from the one used by an existing pretrained model, and you want to pretrain a new model

3. Select the sentences that apply to the BPE model of tokenization.

- ☑ BPE is a subword tokenization algorithm that starts with a small vocabulary and learns merge rules.
- ☐ BPE is a subword tokenization algorithm that starts with a big vocabulary and progressively removes tokens from it.
- ☑ BPE tokenizers learn merge rules by merging the pair of tokens that is the most frequent.
- ☐ A BPE tokenizer learns a merge rule by merging the pair of tokens that maximizes a score that privileges frequent pairs with less frequent individual parts.
- ☑ BPE tokenizes words into subwords by splitting them into characters and then applying the merge rules.
- ☐ BPE tokenizes words into subwords by finding the longest subword starting from the beginning that is in the vocabulary, then repeating the process for the rest of the text.

4. What are the labels in a masked language modeling problem?

- ☑ Some of the tokens in the input sentence are randomly masked and the labels are the original input tokens.
- ☐ Some of the tokens in the input sentence are randomly masked and the labels are the original input tokens, shifted to the left.
- ☐ Some of the tokens in the input sentence are randomly masked, and the label is whether the sentence is positive or negative.
- ☐ Some of the tokens in the two input sentences are randomly masked, and the label is whether the two sentences are similar or not.

5. When should you pretrain a new model?

☑ When there is no pretrained model available for your specific language
☐ When you have lots of data available, even if there is a pretrained model that could work on it
☐ When you have concerns about the bias of the pretrained model you are using

. Is there something wrong with the following code?

```
from transformers import AutoTokenizer, AutoModel

tokenizer = AutoTokenizer.from_pretrained("bert-base-cased") model = AutoModel.from_pretrained("gpt2")

encoded = tokenizer("Hey!", return_tensors="pt") result = model(**encoded)
```

☐ No, it seems correct.
☑ The tokenizer and model should always be from the same checkpoint.
☐ It's good practice to pad and truncate with the tokenizer as every input is a batch.

Getting Your Attention with Self-Attention

Recall that the transformer architecture uses scaled dot-product attention to compute *attention weights*:

$$\boldsymbol{\alpha}^{(t)} = \text{softmax}\left(\frac{\mathsf{q}_t\mathsf{K}^\top}{\sqrt{h}}\right) \in \left[0, 1\right]^n$$

The resulting embedding in the output of attention at position t are:

$$\text{Attention}(\mathsf{Q}, \mathsf{K}, \mathsf{V})_t = \sum_{t'=1}^{n} \alpha_{t'}^{(t)} \mathsf{v}_{t'} \in \mathbb{R}^{1\times h},$$

where $\boldsymbol{\alpha}^{(t)} = \left[\alpha_0^{(t)}, \ldots, \alpha_n^{(t)}\right]$. The same idea can be stated in a matrix form,

$$\text{Attention}(\mathsf{Q}, \mathsf{K}, \mathsf{V}) = \text{softmax}\left(\frac{\mathsf{Q}\mathsf{K}^\top}{\sqrt{h}}\right)\mathsf{V} \in \mathbb{R}^{n\times h}.$$

In the above equations:

- h is the hidden state dimension and n is the input sequence length;
- $\mathsf{X} \in \mathbb{R}^{n\times h}$ is the input to the attention;

- $x_t \in \mathbb{R}^{1\times h}$ is the slice of X at position t, i.e. vector representation (embedding) of the input token at position t;
- $W_q, W_k, W_v \in \mathbb{R}^{h\times h}$ are the projection matrices to build query, key and value representations;
- $Q = XW_q \in \mathbb{R}^{n\times h}$, $K = XW_k \in \mathbb{R}^{n\times h}$, $V = XW_v \in \mathbb{R}^{n\times h}$ are the query, key and value representations;
- $q_t = x_t W_q \in \mathbb{R}^{1\times h}$ is the slice of Q at position t. Similarly, $k_t = x_t W_k \in \mathbb{R}^{1\times h}$ and $v_t = x_t W_v \in \mathbb{R}^{1\times h}$.

Forget 'Softmax', 'Argmax' is the New Boss in Town!

Recall from lectures that we can think about attention as being queryable softmax pooling. In this problem, we ask you to consider a hypothetical 'argmax' version of attention where it returns exactly the value corresponding to the key that is most similar to the query, where similarity is measured using the traditional inner-product. In other words, here use a version of Attention that instead of using softmax we use argmax to generate outputs from the attention layer. For example, `softmax([1, 3, 2])` becomes `argmax([1, 3, 2]) = [0, 1, 0]`.

1. Perform argmax attention with the following keys and values:

$$\text{Keys} = \left\{ \begin{bmatrix} 0 \\ 1 \\ 2 \end{bmatrix}, \begin{bmatrix} 1 \\ 1 \\ 2 \end{bmatrix}, \begin{bmatrix} -1 \\ 1 \\ -2 \end{bmatrix}, \begin{bmatrix} 0 \\ -1 \\ -2 \end{bmatrix} \right\}, \text{Values} = \left\{ \begin{bmatrix} 3 \\ 2 \\ -4 \end{bmatrix}, \begin{bmatrix} 3 \\ -2 \\ 4 \end{bmatrix}, \begin{bmatrix} 3 \\ 2 \\ 0 \end{bmatrix}, \begin{bmatrix} 3 \\ 2 \\ 4 \end{bmatrix} \right\}$$

   using the following query:

$$q = \begin{bmatrix} -2 \\ 1 \\ 1 \end{bmatrix}$$

   What would be the output of the attention layer for this query? Remember, to simplify calculations, use an argmax instead of softmax. Note about notation: each vector in 'Keys = { . }' and 'Values{ . }' are key/value vectors corresponding to four, non-interchangeable positions (i.e., ordering of these vectors matter).

- $q^T k_1 = (-2)(0) + (1)(1) + (1)(2) = 3$
  $q^T k_2 = (-2)(1) + (1)(1) + (1)(2) = 1$
  $q^T k_3 = (-2)(-1) + (1)(1) + (1)(-2) = 1$
  $q^T k_4 = (-2)(0) + (1)(-1) + (1)(-2) = -3$
- $\text{argmax}([3,\ 1,\ 1,\ -3]) = [1,\ 0,\ 0,\ 0]$
- Output $= 1 \cdot v_1 + 0 \cdot v_2 + 0 \cdot v_3 + 0 \cdot v_4 = v_1 = \begin{bmatrix} 3 \\ 2 \\ -4 \end{bmatrix}$

2. How does this design choice affect our ability to usefully train models involving attention? (**Hint:** think about how the gradients flow through the network in the backward pass. Can we learn to improve our queries or keys during the training process?)

    - Argmax is a piecewise constant function so at most points it has slope of 0 and therefore a gradient of 0. This means that most queries will have no gradient so they can't be learned. In contrast, Softmax is a smooth function where every point has some slope and therefore some gradient.

Superposition of Information in Self-Attention

We will show that the attention mechanism is able to copy the information from its input, whenever needed.[1]

1. **'Copying' mechanism in self-attention:** We'll first show that it is particularly simple for attention to "copy" a value vector to the output. Describe (in 1-2 sentences) what properties of the inputs to the attention operation would result in the output $\text{Attention}(Q, K, V)_t$ being approximately equal to v_j for some $j \in [0 \ldots n]$.
   Hint: Show that there can exist $j \in [0 \ldots n]$ such that $\alpha_j^{(t)} \gg \sum_{i \neq j} \alpha_k^{(t)}$, if certain property holds for the query q_t, and the keys $\{k_0 \ldots k_n\}$.

    - If the query Q_t has a much larger inner product with one of the keys K_j than with any other key then Softmax puts almost all of the weight on position j. The output is approximately V_j; essentially copying vector V_j.

[1] Question credit: John Hewitt

2. **Extracting signals after averaging them:** Instead of focusing on just one vector v_j, attention mechanism might want to incorporate information from multiple source vectors. Consider the case where we instead want to incorporate information from two vectors v_a and v_b, with corresponding key vectors k_a and k_b. How should we combine information from two value vectors v_a and v_b? A common way to combine values vectors is to average them: $\bar{v} = \frac{1}{2}(v_a + v_b)$. However, after such averaging it is not quite clear how to tease apart the original information in value vectors v_a and v_b. Unless ... some special properties hold! Suppose that although we don't know v_a or v_b, we do know that v_a lies in a subspace[2] A formed by the m basis vectors[3] $\{a_1, a_2, \ldots, a_m\}$, while v_b lies in a subspace B formed by the p basis vectors $\{b_1, b_2, \ldots, b_p\}$. This means that any v_a can be expressed as a linear combination of its basis vectors, as can v_b. All basis vectors have norm 1 and are orthogonal to each other. Additionally, suppose that the two subspaces are orthogonal; i.e. $a_j^\top b_k = 0$ for all j, k. Using the basis vectors $\{a_1, a_2, \ldots, a_m\}$, construct a matrix M such that for arbitrary vectors $v_a \in A$ and $v_b \in B$, we can use M to extract v_a from the average vector $\bar{v}$. In other words, we want to construct M such that for any v_a, v_b, $M\bar{v} = v_a$. Show that $M\bar{v} = v_a$ holds for your M.

   - We need M to be a matrix that keeps any vectors in subspace A and suppresses anything in subspace B. We also need it to reverse the $\frac{1}{2}$ from the averaging operation. Since the m basis vectors of A are orthogonal by definition, taking the dot product of one of those basis vectors $a_j^T v_a$ gives us the coefficient c_j which is the coordinate of v_a along that basis vector. We can multiply this coefficient by a_j to return that component back into the original space as $a_j(a_j^\top v_a) = c_j a_j$.
   - The matrix $a_j a_j^T$ is the projection onto the single direction a_j.
   - Applying this to all the basis vectors and summing gives us

$$\sum_{j=1}^{m} a_j a_j^\top v_a = \sum_j c_j a_j = v_a$$

[2] https://en.wikipedia.org/wiki/Linear_subspace

[3] https://en.wikipedia.org/wiki/Basis_(linear_algebra)

- The matrix $P = \sum_i a_j a_j^T = AA^T$ is the orthogonal projector onto A and acts as an identity on A.
- We are given that the subspace B is orthogonal to the subspace A. Since v_b is a linear combination of the basis vectors of B then v_b is orthogonal to the basis vectors of A. So $Pv_b = 0$.
- Applying P to $\bar{v}$ gives us only $\frac{1}{2}v_a$ because of the halving part of the averaging operation. To correct for this we simply multiply P by 2 giving us matrix $M = 2P = 2AA^T = 2\sum_{j=1}^{m} a_j a_j^T$.
- We can apply M to the average $M\bar{v} = 2\sum_j a_j a_j^\top \cdot \frac{1}{2}(v_a + v_b) = \sum_j a_j a_j^\top v_a + \sum_j a_j a_j^\top v_b$. Since $a_j a_j^\top v_b = 0$ for every j we are left only with the left side of the sum which we already showed reconstructs v_a.

3. **Averaging pairs of representations:** As before, let v_a and v_b be two value vectors corresponding to key vectors k_a and k_b, respectively. Assume that (1) all key vectors are orthogonal, so $k_i^\top k_j = 0$ for all $i \neq j$; and (2) all key vectors have norm 1. Find an expression for a query vector q_t such that $\text{Attention}(Q, K, V)_t = \frac{1}{2}(v_a + v_b)$ and justify your answer.

   More here: https://web.stanford.edu/class/cs224n/assignments/a5.pdf

   - We need a query vector with points along k_a and k_b such that it scores equally and highly on both of those keys. Since the key vectors are orthogonal to each other each key contributes only to it's own score with no cross-contribution amongst subspaces. A query vector that is a linear combination of these two key vectors would satisfy this requirement, however, to ensure Softmax sufficiently concentrates the score on these two keys (assuming there are other keys) we need to scale this linear combination by some sufficiently large factor β. If there are no other key vectors than β can be arbitrary or just 1. The expression for this query vector is $q_t = \beta\,(k_a + k_b)$.
     - Scoring each key vector with this query vector: $q_t^\top k_a = \beta\,(k_a^\top k_a + k_b^\top k_a) = \beta k_a k_a^T \qquad q_t^\top k_b = \beta k_b k_b^T,$

Extra Credit: Importance of 'Scaling' in Self-Attention

In practice, we scale each dot product $QK^\top$ by a factor of $\sqrt{h}$. This is called *scaled dot product attention*. In this part, we will prove why we perform this scaling. Suppose we are performing a dot product between a key k and query q, where k, q $\in \mathbb{R}^{1\times h}$ and k, q $\sim \mathcal{N}\left(\boldsymbol{\mu}, \sigma^2\mathbf{I}\right)$. You can assume that k, q are sampled independently.

1. Compute $\mathbf{E}\left[\mathrm{qk}^\top\right]$ in terms of $\boldsymbol{\mu}, h, \sigma$.
   - $qk^\top = \sum_{i=1}^{h} q_i k_i,$
   - We are given that q and k vectors are each random drawn from a Gaussian distribution. Since the vectors are sampled independently, their entries are independent across dimensions, therefore qk^T is the sum of h independent terms.
   - If every component of q and k has the same mean μ then the expectation of each component of qk^T is μ^2.
   - $\mathbb{E}\left[qk^\top\right] = \sum_{i=1}^{h} \mathbb{E}[q_i]\,\mathbb{E}[k_i] = \sum_{i=1}^{h} \mu_i^2 = \mu\mu^\top = \|\mu\|^2$

2. Compute $\mathrm{Var}\left[\mathrm{qk}^\top\right]$ in terms of $\boldsymbol{\mu}, h, \sigma$.
   - Each term h_i has independent factors q_i and k_i.
     - Since $\mathrm{Var}(x) = \mathbb{E}[x^2] - (\mathbb{E}[x])^2$ then $\mathrm{Var}(q_i k_i) = \mathbb{E}[q_i^2]\,\mathbb{E}[k_i^2] - \left(\mathbb{E}[q_i]\,\mathbb{E}[k_i]\right)^2 = (\sigma^2 + \mu_i^2)^2 - \mu_i^4 = \sigma^4 + 2\sigma^2\mu_i^2$
   - The terms are independent so the variances add :
     - $\mathrm{Var}\left[qk^\top\right] = \sum_{i=1}^{h} \left(\sigma^4 + 2\sigma^2\mu_i^2\right) = h\sigma^4 + 2\sigma^2\|\mu\|^2$
   - Since they individual terms have a shared mean μ this becomes
     - $\mathrm{Var}\left[qk^\top\right] = h\sigma^2\left(\sigma^2 + 2\mu^2\right)$

3. Based on the variance computed above and assuming that $\boldsymbol{\mu} = 0$ and $\sigma = 1$, explain why we need to scale the dot product by $\sqrt{h}$. Explain why this scaling is important for numerical stability.

   - Given $\mu = 0$ and $\sigma = 1$ we have:
     - $\mathbb{E}\left[qk^\top\right] = 0, \qquad \text{Var}\left[qk^\top\right] = h, \qquad \text{std} = \sqrt{h}$
   - Without scaling the logins have standard deviation of $\sqrt{h}$ so for a large h individual scores will be far from 0. With large magnitude logics, softmax becomes close to one-hot with one weight nearly equal to 1 and others basically 0. In this case virtually 0 gradient flows to the queries and keys so training stalls. Large logits can also make the the exp function overflow. Keeping the logins at unit variance keeps softmax inputs in a “well-behaved” range and the weights smoothly distributed.
     - Scaling by $\dfrac{1}{\sqrt{h}}$ gives unit variance regardless of the value of h
     - $\text{Var}\left[\dfrac{qk^\top}{\sqrt{h}}\right] = \dfrac{1}{h} \cdot h = 1$

Programming

In this programming homework, we will finetune encoder LMs for a classification task with yes/no questions. In particular, we will

- Implement full-parameter finetuning.
- Explore various parameter-efficient finetuning strategies.

Skeleton Code and Structure:

The code base for this homework can be found at this GitHub repo under the `hw6` directory. Your task is to fill in the missing parts in the skeleton code, following the requirements, guidance, and tips provided in this pdf and the comments in the corresponding .py files. The code base has the following structure:

- `base_classification.py`: implements the basic full finetuning
- `classification.py`: implements different parameter-efficient finetuning

TODOs — Your tasks include 1) generate plots and/or write short answers based on the results of running the code; 2) fill in the blanks in the skeleton to complete the code. We will explicitly mark these plotting, written answer, and filling-in-the-blank tasks as

TODOs in the following descriptions, as well as a `# TODO` at the corresponding blank in the code.

Submission:

Your submission should contain two parts: 1) plots and short answers under the corresponding questions below; and 2) your completion of the skeleton code base, in a `.zip` file, which includes the two provided skeleton files and a `openai_gpt-boolq.py` script you created (details in 3.3).

Overview and Dataset

In this programming assignment, you will get your hands dirtier with building self-supervised models. You will build a classifier using the Huggingface Transformer library and PyTorch. This classifier is expected to solve the BoolQ dataset . You can find this dataset on Huggingface's dataset hub: https://huggingface.co/datasets/boolq. In this task, each input consists of a question statement and a passage that may contain the answer to this question. The output is the answer to the question which may be one of 'Yes' or 'No' labels. Here is an example:

"passage": "Äll biomass goes through at least some of these steps: it needs to be grown, collected, dried, fermented, distilled, and burned...", "question": "does ethanol take more energy make that produces" "answer": false,

The data comes with training and validation subsets. We will select a subset of the training data as 'test' set. So for the main experiments we will use 8k instances for training and the rest for testing.

Python Scripts Arguments

Many of the **TODOs** below involves setting different training configurations and hyper-parameters, this could be achieved through the arguments provided in `main` of both `base_classification.py` and `classification.py` - they should be self-explanatory.

General Hint

Follow the specifications and requirements in the code comments to complete missing lines.

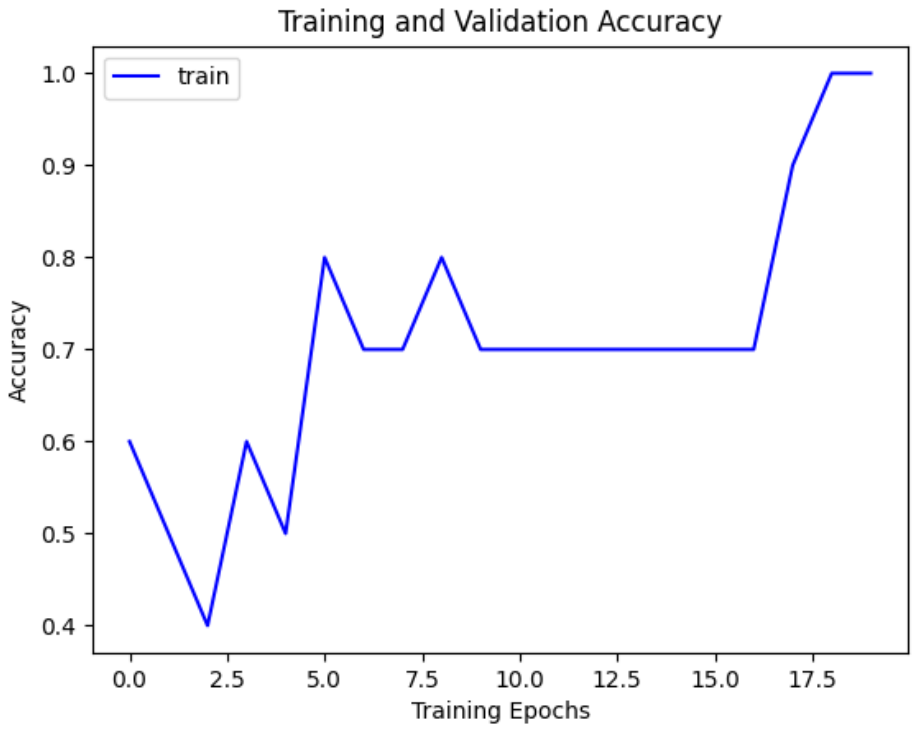
Finetuning: Basics

1. Let's start with the basic full finetuning, where all the parameters will be updated during the training. Specifically, we will use `AutoModelForSequenceClassification` which implements a classifier that maps the representation of encoder LMs (BERT, RoBERTa, etc.) into a fixed set of labels. Use this class to build a binary classifier using the representations of `distilbert-base-uncased` . The starter code for full finetuning in `base_classification.py` has a bunch of basic functionalities implemented, but

it is missing details.
TODOs: Read and complete the missing lines in `evaluate_model` and `train` functions in `base_classification.py` to complete the functionalities of full finetuning.

To make sure your implementation is correct, we can train the model on 10 training instances for a few epochs (say, a total of 20 epochs). Note that over-fitting a few training examples is a good first test to make sure your implementation can successfully learn the patterns of your data. You should be able to complete the starter code and run it on a few instances in your local machine. You should be able to see that your classifier successfully over-fits its few training examples.
TODOs: Run `base_classification.py` with the following command on your local machine, paste the generated plot of training accuracy vs. epochs plots below.

> python base_classification.py –small_subset –device cuda –model "distilbert-base-uncased" –batch_size "64" –lr 1e-4 –num_epochs 20



2. Now, rather than running training on a subset of the data, try training with all the data on your compute environment.

   TODOs Train for 30 epochs and paste the generated plot of train and dev accuracies vs. epochs below. Also report the largest batch size.
   Hint: You should see that training accuracy goes high, while the dev accuracy will go high and then turn downhill as the model overfits the data.



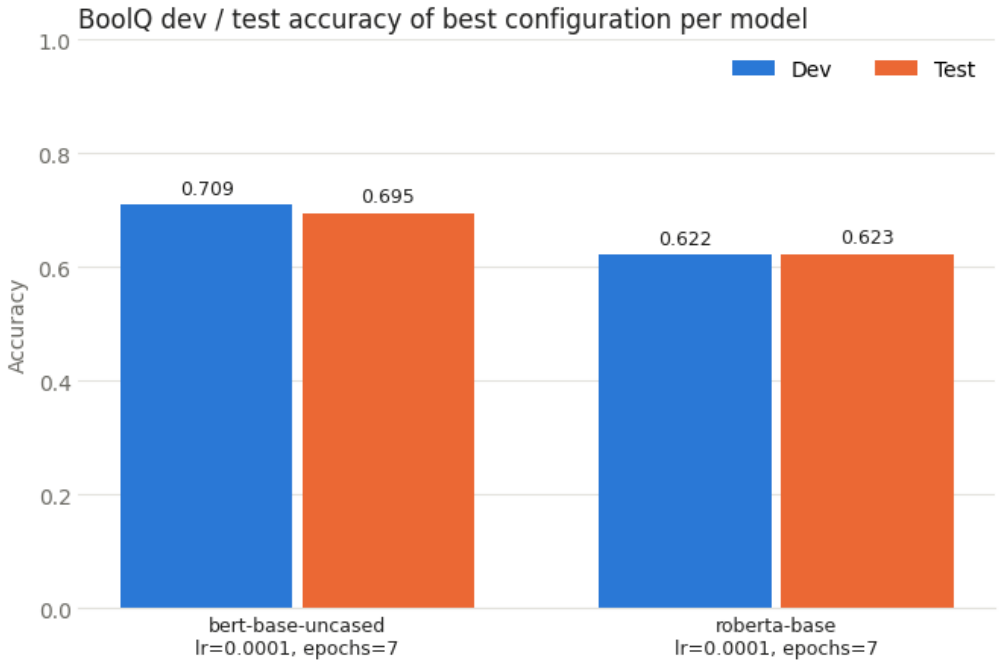
3. Let's implement hyper-parameter selection by exploring the combination of different hyper-parameter choices.

   TODOs: Train models for various choices of learning rates (`1e-4, 5e-4, 1e-3`) and training epochs (`7, 9`). Select the model with the highest accuracy on the dev set and report its accuracy on the test set. Also report the corresponding choice of hyperparameters

   - After running the grid-search on these hyperparameters, the model with the best performance was using a learning rate of 0.0001 and was trained over 7 epochs. Below are the findings of the grid-search.

	lr	epochs	dev_acc
0	0.0001	7	0.705810
1	0.0001	9	0.682569
2	0.0005	7	0.621713
3	0.0005	9	0.621713
4	0.0010	7	0.621713
5	0.0010	9	0.621713

4. **TODOs** Now repeat the previous experiment with other models. In particular, create a bar plot that shows test/dev accuracies for two of your favorite of the following models: `distilBERT-base-uncased, BERT-base-uncased, BERT-large-uncased, BERT-base-cased, BERT-large-cased, RoBERTa-base, RoBERTa-large`. Note, it is possible that some of these models wouldn't fit in your compute environment for any choice of batch size, in which case report 0 for their performance.

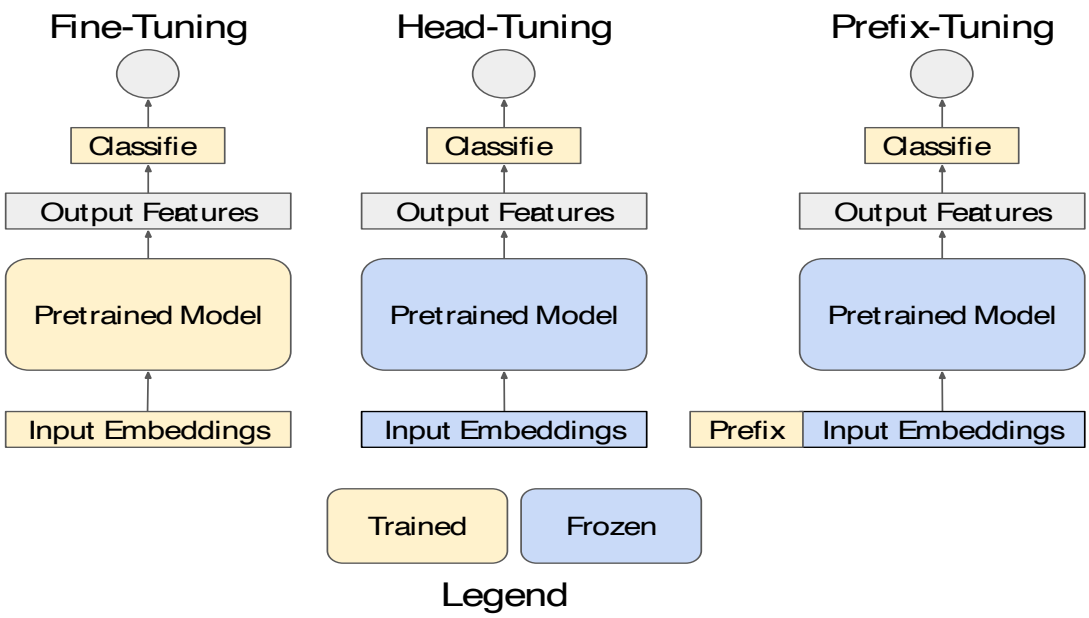


Parameter-Efficient Fine-Tuning

In this section, we will implement two more efficient ways of tuning language models: **Head-Tuning**, which freezes all parameters but the classification head, and **Prefix-Tuning**, which also freezes all parameters but the classification head, and additionally appends a fixed length ℓ of trainable embeddings to the input embeddings. See figure 1 for an illustration of which parameters are getting updated in each fine-tuning method we will explore in this homework.

Note that “output features" tensor for a single sequence of input should be of shape $\big(\text{length}, \text{hidden size}\big)$, or equivalently $\mathbb{R}^{N\times d}$. However, the classifier expects a tensor of shape $\mathbb{R}^{1\times d}$. There are multiple ways to transform the output features to $\mathbb{R}^{1\times d}$, but in this work, we will take the **averaged** token representation.

Instead of using `AutoModelForSequenceClassification`, we now use the base `AutoModel` class to tailor the model to our needs. But don’t worry, a skeleton code is provided to you in `classification.py`, you would only need to modify the details.



Overview of fine-tuning methods. Blue indicates parts that are frozen (no gradient update) during the fine-tuning process. Yellow indicates parts that are trained (with gradient updates).

1. For comparison, let's redo the full finetuning first. Change the base model to `RoBERTa-base`. Implement full fine-tuning by adding a binary classifier on the averaged sentence representation.

   TODOs: Read and complete the missing lines

   a. in `evaluate_model` function

   b. after `for index, batch in tqdm(enumerate(train_dataloader)):` in `train` function

   in `classification.py` to complete the functionalities of full finetuning.
   Hint: Your implementation should be the same as that of `base_classification.py` for `evaluate_model` and almost the same except for how we get the logtis for `train` (until loss backward).
   TODOs: Read and complete the missing lines after `if self.type == 'full'` in the `forward` function of the `CustomModelforSequenceClassification` class in `classification.py`. Run the code with `type="full"` to train the model with full finetuning. Report the final accuracy you obtained along with the hyper-parameters you used (number of epochs and learning rate).

   - Roberta-base model with lr=1e-4, epochs=7
   - Dev accuracy 0.6217; test accuracy 0.6356

2. **TODOs**: Write down the number of tuned parameters for head-tuning for BoolQ, given that the model hidden size is d.

   - BoolQ has only 2 labels. In RoBERTa-base d=768 so there are $2 \times 768 = 1{,}538$ parameters.

   Now let's implement head-tuning by adding a binary classifier on the averaged sentence representation.
   TODOs: Read and complete the missing lines after `if self.type == 'head'` in the `forward` function of the `CustomModelforSequenceClassification` class in `classification.py`. You would also need to modify the optimizer by adding code after `mymodel.type == "head":` in the `train` function in `classification.py`. Report the final accuracy on full head-tuning along with the hyper-parameters you used[4].

   - Roberta-base with lr=1e-4 and epochs=7
   - Dev accuracy = 0.6214; Test accuracy = 0.6132

3. **TODOs**: Write down the number of tuned parameters for our prefix-tuning on BoolQ that appends trainable prefix embeddings of length 128, given the model hidden size is d.
   - The number of tuned parameters for RoBERTa-base would be the size of the prefix + 2 biases (one per label) multiplied by the hidden size of the model. For RoBERTa-base that is $130 \times 768 = 99{,}842$ tuned parameters.
   It's time to implement prefix tunning!
   TODOs: Read and complete the missing lines after `if self.type == 'prefix'` in the `forward` function of the `CustomModelforSequenceClassification` class in `classification.py`. You would also need to modify the optimizer by adding code after `mymodel.type == "prefix":` in the `train` function in `classification.py`. Report the final accuracy of prefix-tuning along with the hyper-parameters you used [5]..

   - Roberta-base; lr=1e-4; epochs=7
   - Dev accuracy = 0.6217; Test accuracy = 0.6321

---

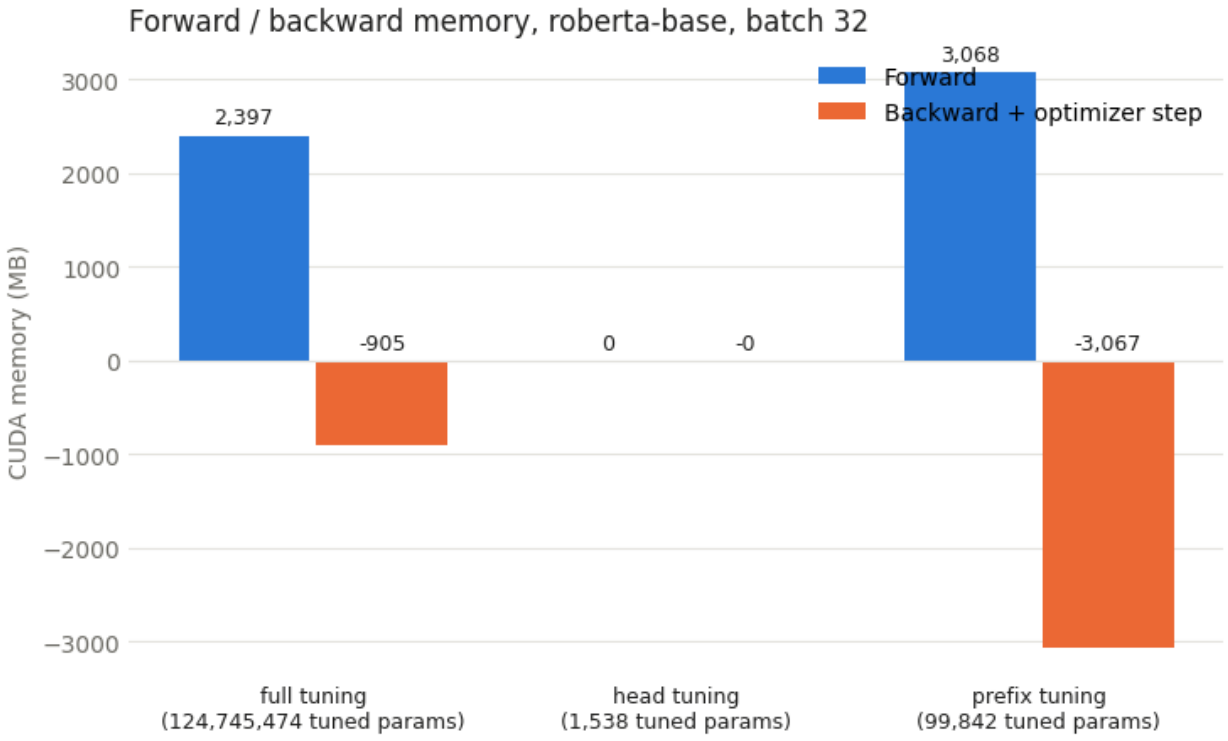
[4] You would be only graded by the correctness of your implementation, but if you want good performance, Try using a larger learning rate and longer training epochs.

[5] Again, you would be only graded by the correctness of your implementation, but if you want good performance, Try using a larger learning rate and longer training epochs.

4. **Extra Credit:** Here is a simple tutorial on how to use torch.profile to record the memory usage of forward and backward operations.

   TODOs Generate a bar plot of the forward/backward memory usage of full-finetuning, head-tuning, and prefix-tuning. Note that you don't need to upload any code for this.

   - I ran this on one batch of 32 real BoolQ examples with the same model class, optimizer setup and loss as in training. The number for a block is the memory allocated minus the memory freed during it so backward values can be negative.
   - Prefix tuning's forward pass used the most memory despite having far fewer tuned params compared to full tuning. This is thanks to its 256-token inputs. Full fine-tuning came in next highest because it stored gradients plus AdamW states for all of RoBERTa with over 124mm tuned params. Head tuning was by far the smallest with only 1.5k tune params.



5. **LoRA** fine-tuning decomposes the parameter updates of a large dense matrix to two low-rank matrices. For a matrix M of size $\mathbb{R}^{d\times d}$, the full updates require storing the gradients and optimizer states of $d \times d$ parameters.
   TODOs: However, if we decompose the gradient update on the full matrix $\Delta M \approx AB$ where $A \in \mathbb{R}^{d\times n}$ and $B \in \mathbb{R}^{n\times d}$, write down the number of parameters whose gradients and optimizer states we need to store:

   - Only A and B get gradients and optimizer states while M remains frozen. Since A is a $d \times n$ matrix and B is a $n \times d$ matrix we would need to store $2dn$ parameters with gradients and optimizer states.
     .

6. **Extra Credit:** Implement **LoRA** fine-tuning.
   TODOs: Create `classification_lora.py` and copy `classification_base.py` to it and implement the LoRA fine-tuning method using the peft library. This should be just changing a few lines of code. Report the trainable parameters and final accuracy of your LoRA fine-tuning, as well as your hyper-parameters.

   - I ran LoRA fine-tuning on a RoBERTa-base model with lr=1e-4 and epochs=7.
   - Dev accuracy 0.72813; Test accuracy 0.73721; a meaningful improvement to prefix-tuning.

In-Context Learning

1. Here we will prompt language models.[6] You should sign up for the OpenAI API, which lets you use GPT-3 a large, neural language model like the ones that we learned about in lecture.

   The OpenAI API is a paid service. OpenAI will give you a few dollars in credit when you first create your account (See the usage page). For this assignment, the cost should be less than that. For the first part of the assignment, we'll get warmed up by playing with the OpenAI API via its interactive Playground website. Later we'll see how to integrate it directly into our code.

   OpenAI

   OpenAI products offer a wide spectrum of capabilities and modes of interaction with their models. For this homework, **we use the `Completion` mode for both playground and API calls**. In the future, you can explore other modes that might be more up-to-date and powerful, e.g. `Chat`(Playground, APIs), `Assistants` (Playground, APIs), and many more!
   Note: we suggest you to prioritize your credit usage on finishing this homework

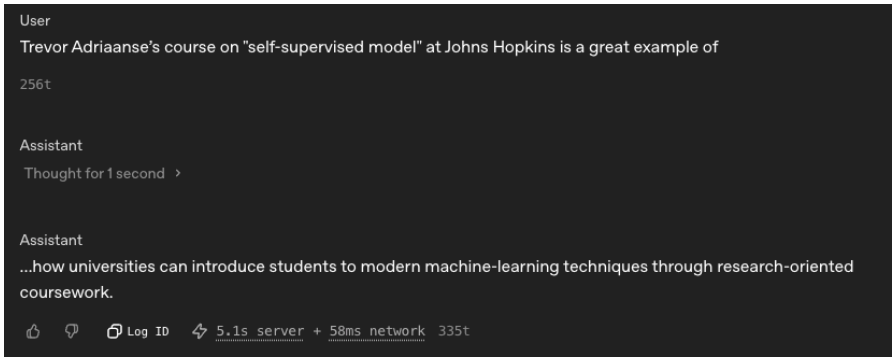
[6] Question credit: Chris Callison-Burch's AI course at UPenn.

question with `Completion` before exploring other modes, as querying powerful models might be more expensive.
First, let's learn some basic terminology:

- Prompt: the input to the model
- Completion: what the model outputs

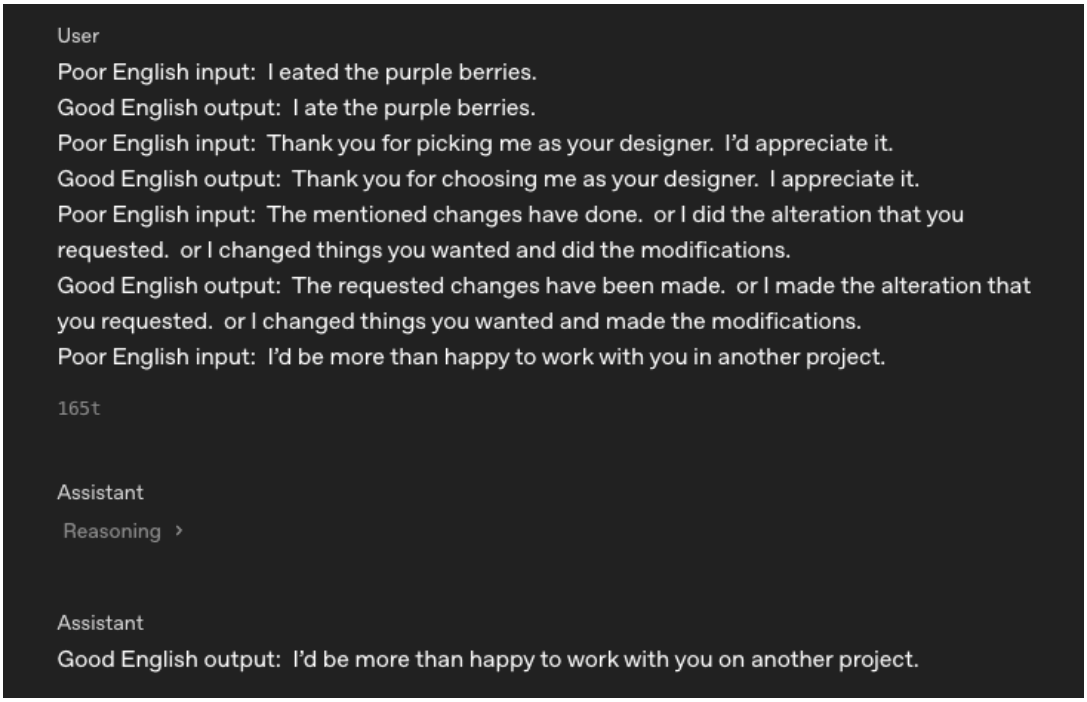
Let's try it out.
TODOs: Paste this prompt into the playground, press the "Generate" button, and see what it says. Paste a screenshot of the completion below:

> Note - completion mode was deprecated so I just made a note in the model instructions that it should complete the prompt.
>
> Also please note the prompts were missing from the editable `.docx` file so I copied them from the pdf but did't paste them here to save space.



Now save its output for the end of the semester for your course reviews ... just kidding!!

2. There are several controls on the right hand side of the playground. These are:
    - **Model:** GPT models come in several different sizes and capabilities. You can read more documentation on these models here. For our homework, we use the `gpt-3.5-turbo-instruct` model.
        - This model has been removed. To keep it on an efficient model I just left it as GPT-6-Luna. Most of the other controls do not appear to be available in the playground any longer also.
    - **Temperature and Top P sampling:** control how the model samples tokens from its distribution. Setting Temperature to 0 will cause the model to produce the highest probability output. Setting it closer to 1 will increase its propensity to create more diverse output.



- **Maximum length:** what's the maximum length (in tokens) that the model will output?
- **Stop sequences:** you can specify what tokens should cause the model to stop generating. You can use the newline character or any special sequence that you designate.
- **Frequency Penalty and Presence Penalty:** two parameters that help to limit how much repetition there is in the model's output.
- **Best of:** Best of n indicates the model generate n completion candidates on the server side, and display the top-1 on the client (user) side.

For the remaining parts of this section, set the engine to `davinci` so that it uses vanilla language model (no additional tuning with human feedback).
In-context learning: In addition to writing awesome reviews of your professors, you can design prompts to get GPT-3 to do all sorts of surprising things. For instance, GPT-3 can perform few-shot learning. Given a few examples of a task, it can learn a pattern very quickly and then be used for classification tasks. It often helps to tell the model what you want it to do.

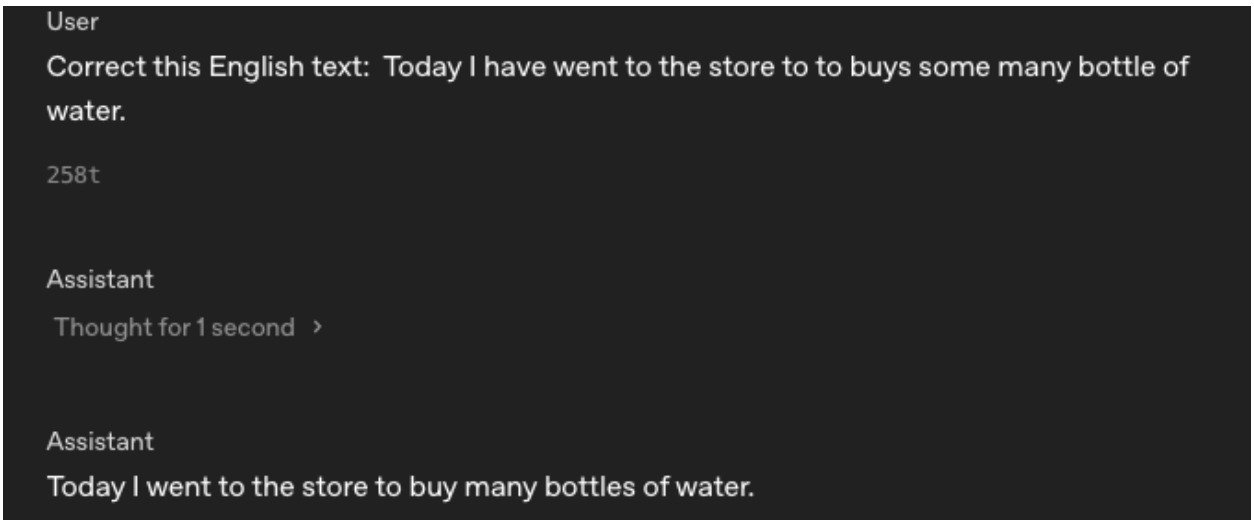
Here's an example from the paper that introduced GPT-3. It shows a few-show learning example for correcting grammatically incorrect English sentences.

TODOs: Paste this prompt into the playground, press the "Generate" button, and see what it says. Paste the completion below:

3. **Instruction following:** In addition to few shot learning, GPT and other large language models do a pretty remarkable job in “zero-shot” scenarios. You can give them instructions in natural language and they may produce remarkably good output.

   For example, let’s try the following prompt:

   TODOs: Paste this prompt into the playground, press the “Generate” button, and see what it says. Paste the completion below:



4. **Programmatic access:** In addition to prompting GPT models in the playground, you can write Python code to query models with provided OpenAI APIs. Now let’s use few-shot prompting to solve a few instance of BoolQ.
   TODOs: Write a Python script for few-shot prompting OpenAI GPT models on BoolQ. Specifically, form a prompt by randomly selecting 8 demonstrations from BoolQ. Make sure to have a balanced prompt (50% ‘yes’s and the rest ‘no’s) and interleave them to prevent the recency and label-imbalance biases. Using this in-context demonstration, evaluate `gpt-3.5-turbo-instruct` on a small subset of BoolQ instances (say, 30 examples). Hand in your code used to perform this evaluation, it should be named as `openai_gpt-boolq.py`. Write down the evaluation accuracy below.
   Hints:
   - We use the `gpt-3.5-turbo-instruct` model (`model="gpt-3.5-turbo-instruct"`)

    - You can use the playground to create a skeleton code to start with based on a prompt that you can then use in your Python projects. Click on the “View Code” button on the top-right, and you’ll get some code that you can convert into a script.
    - You can also refer to the full documentation for creating completion and https://platform.openai.com/docs/api-reference/completions/object.
    - Using OpenAI API requests an API key. You can generate a key associated with your account here. To input the key, you can either do `export OPENAI_API_KEY=<your key>` on your compute environment before running the script. Or refer to the documentation.

- As mentioned above, gpt-3.5-turbo-instruct was retired. I decided to use gpt-4.1-mini which his the next most similar model. This required a change to the endpoint from `completions` to `chat.completions`. I was using Claude to help me write the code which suggested also using Qwen2.5-7B-Instruct since it is also an instruction-tuned model.
- Accuracy: 27/30 = 0.900

Optional Feedback

Have feedback for this assignment? Found something confusing? We’d love to hear from you!